

LAPORAN PRAKTIKUM INTERNET OF THINGS 2



Oleh:

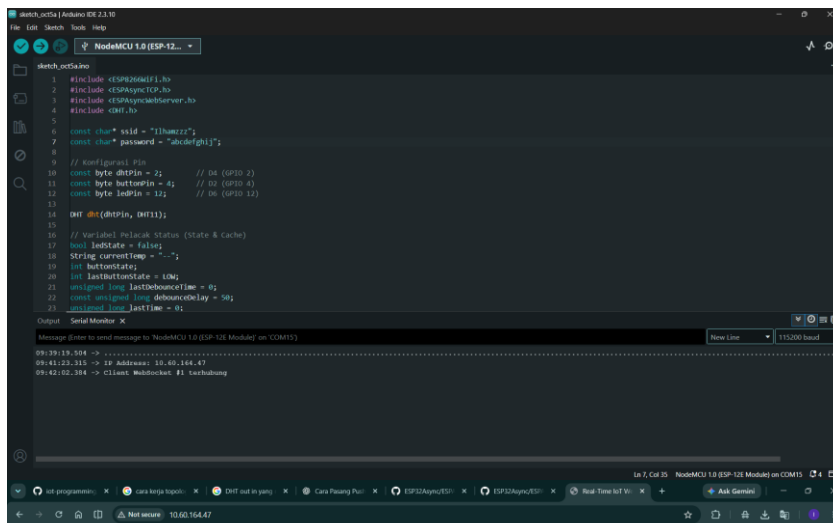
Nama : Ilham Alian Safaat
Npm : 25782012
Kelas : 3A

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.

**TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2026/2027**

4.5.1 Praktikum 1: *Real-Time Dashboard & Sinkronisasi Tombol Fisik*



```
1 #include <ESP2866WiFi.h>
2 #include <ESPAsyncTCP.h>
3 #include <ESPAsyncWebServer.h>
4 #include <DHT.h>
5
6 const char* ssid = "11hamzz";
7 const char* password = "shadefghij";
8
9 // konfigurasi pin
10 const byte dhtPin = 2; // D4 (GPIO 2)
11 const byte buttonPin = 4; // D2 (GPIO 4)
12 const byte ledPin = 12; // D6 (GPIO 12)
13
14 DHT dht(dhtPin, DHT11);
15
16 // Variabel pelacak status (state & cache)
17 bool ledState = false;
18 String currentTemp = "-.-";
19 int buttonState;
20 int lastButtonTime = 0;
21 unsigned long lastDebounceTime = 0;
22 const unsigned long debounceDelay = 50;
23 unsigned long lastTime = 0;
```

Output

Message given to send message to NodeMCU 1.0 (ESP-12E Module) on COM15

09:39:19.304 -> 09:39:19.304 -> IP Address: 192.168.1.47

09:41:23.315 -> IP Address: 192.168.1.47

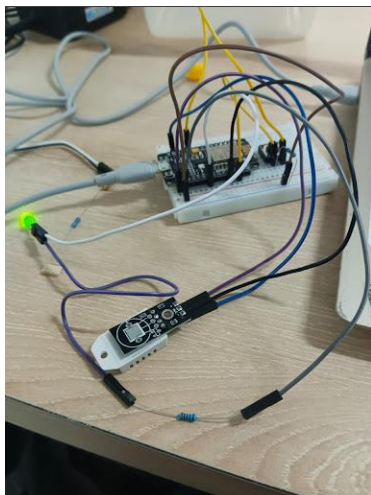
09:42:02.384 -> Client: Multitouch #1 terhubung

Smart Room

Suhu: 2.10 Celcius

LED: ON

Turn OFF



Langkah Kerja Rangkaian:

1. Hubungkan kaki **Anoda** (kaki panjang) LED ke pin **D6 (GPIO 12)**. Hubungkan kaki **Katoda** (kaki pendek) LED ke salah satu kaki **Resistor 220 Ohm**, lalu hubungkan kaki resistor lainnya ke pin **GND NodeMCU** (*sama seperti rangkaian LED pada Modul 3*).

2. Hubungkan pin **DATA / OUT** DHT ke pin **D4 (GPIO 2)**. (VCC DHT ke 3V3, GND ke GND).
3. Pasang **Push Button** pada *breadboard*, melintasi parit tengah. Hubungkan salah satu kakinya ke jalur **3V3 (3.3V)**. Hubungkan kaki seberang tombol ke pin **D2 (GPIO 4)** NodeMCU. Pada titik kaki yang terhubung ke pin D2 tersebut, pasang salah satu kaki **Resistor 10k Ohm**, lalu hubungkan ujung kaki resistor lainnya ke jalur **GND** (*konfigurasi resistor pull-down eksternal seperti pada Modul 1*).

Dan hasil rangkaian akan membuat **Sinkronisasi Dua Arah (Real-Time)**: Mengontrol status lampu LED fisik baik melalui tombol fisik (*push button*) di *breadboard* maupun melalui tombol di antarmuka web *smartphone/laptop* secara bersamaan.

4.6 Latihan

- **Latihan 1 (Teori Jaringan):** Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi *WebSockets* jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode *Meta Refresh HTTP* HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

Kalau pakai Meta Refresh HTTP, browser bakal minta seluruh halaman web (HTML, CSS, header) secara berulang-ulang. Nah, WebSockets itu beda ia cuma ngebuka satu jalur koneksi yang selalu terhubung. Jadi begitu ada pembaruan, data yang dikirim cuma teks angka JSON yang ukurannya sangat kecil tanpa *overhead* header dan kalau layer yang tidak berkedip karena HTTP Meta Refresh itu sistemnya *reload* ulang satu halaman penuh dari awal.

- **Latihan 2 (Analisis Waktu/Timer):** Apa bahaya sistematis yang akan terjadi pada komunikasi *WebSockets* (reaksi antara *browser* dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Fungsi `delay(3000)` itu sifatnya *blocking* (menghentikan kerja CPU selama 3 detik). Akibat yang akan terjadi, NodeMCU bakal "mati" sementara nggak merespons ketika tombol fisik ditekan, terlambat menerima perintah dari web, dan koneksi WebSockets bisa mendadak putus (*timeout*) karena fungsi pembersihan klien (`ws.cleanupClients()`) ikutan macet.

- **Latihan 3 (Modifikasi Front-End):** Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML () apa saja yang menjadi target injeksi/pengubahan teks oleh *script* yang ditulis oleh Anda?

Yang diubah:

tempValue → Elemen untuk mengganti angka suhu.

ledStatus → Elemen untuk mengganti teks status LED (ON atau OFF).

- **Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan):** Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (*temperature*). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (*humidity*) secara *real-time* via WebSockets:
 - Sisi Back-End (C++):** Buat variabel penampung (misal: String currentHum = "--");. Baca nilai kelembapan menggunakan fungsi dht.readHumidity() di dalam siklus timer *non-blocking* millis(). Perbarui fungsi notifyClients() agar menyisipkan data kelembapan ke dalam paket JSON yang dikirim ke *browser* (contoh format: {"led":"...", "suhu":"...", "hum":"..."}).
 - Sisi Front-End (HTML & JavaScript):** Tambahkan elemen kartu (<div class="card">) baru pada kerangka HTML sebagai penampil kelembapan (misal: <h2>Kelembapan: -- %</h2>). Kemudian, modifikasi fungsi JavaScript onMessage(event) agar mengekstrak data kelembapan dari objek JSON dan memperbarui teks pada elemen humValue.

Tuliskan baris kode modifikasi Anda (pada sisi HTML, JavaScript, dan C++)!

1. Sisi Back-End (C++)

- Deklarasikan variabel global kelembapan:

C++

String currentTemp = "--";

String currentHum = "--"; // Penampung data kelembapan

- Update fungsi notifyClients() buat nyisipin data kelembapan ke JSON:

C++

void notifyClients() {

String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", \"";

```

jsonString += "\"suhu\": \"" + currentTemp + "\", ";
jsonString += "\"hum\": \"" + currentHum + "\"}";
ws.textAll(jsonString);
}

```

- Update pembacaan sensor di loop():

C++

```

if ((millis() - lastTime) > 3000) {
    float t = dht.readTemperature();
    float h = dht.readHumidity(); // BACA KELEMBAPAN

    if (!isnan(t) && !isnan(h)) {
        currentTemp = String(t);
        currentHum = String(h);
        notifyClients();
    }

    lastTime = millis();
}

```

2. Sisi Front-End (HTML & JavaScript)

- Menambahkan kartu tampilan baru

HTML

```

<div class="card">
    <h2>Kelembapan: <span id="humValue">--</span> %</h2>
</div>

```

- Update fungsi onMessage di JavaScript:

```

function onMessage(event) {
    var dataObj = JSON.parse(event.data);

    if(dataObj.suhu !== undefined) {
        document.getElementById('tempValue').innerHTML = dataObj.suhu;
    }

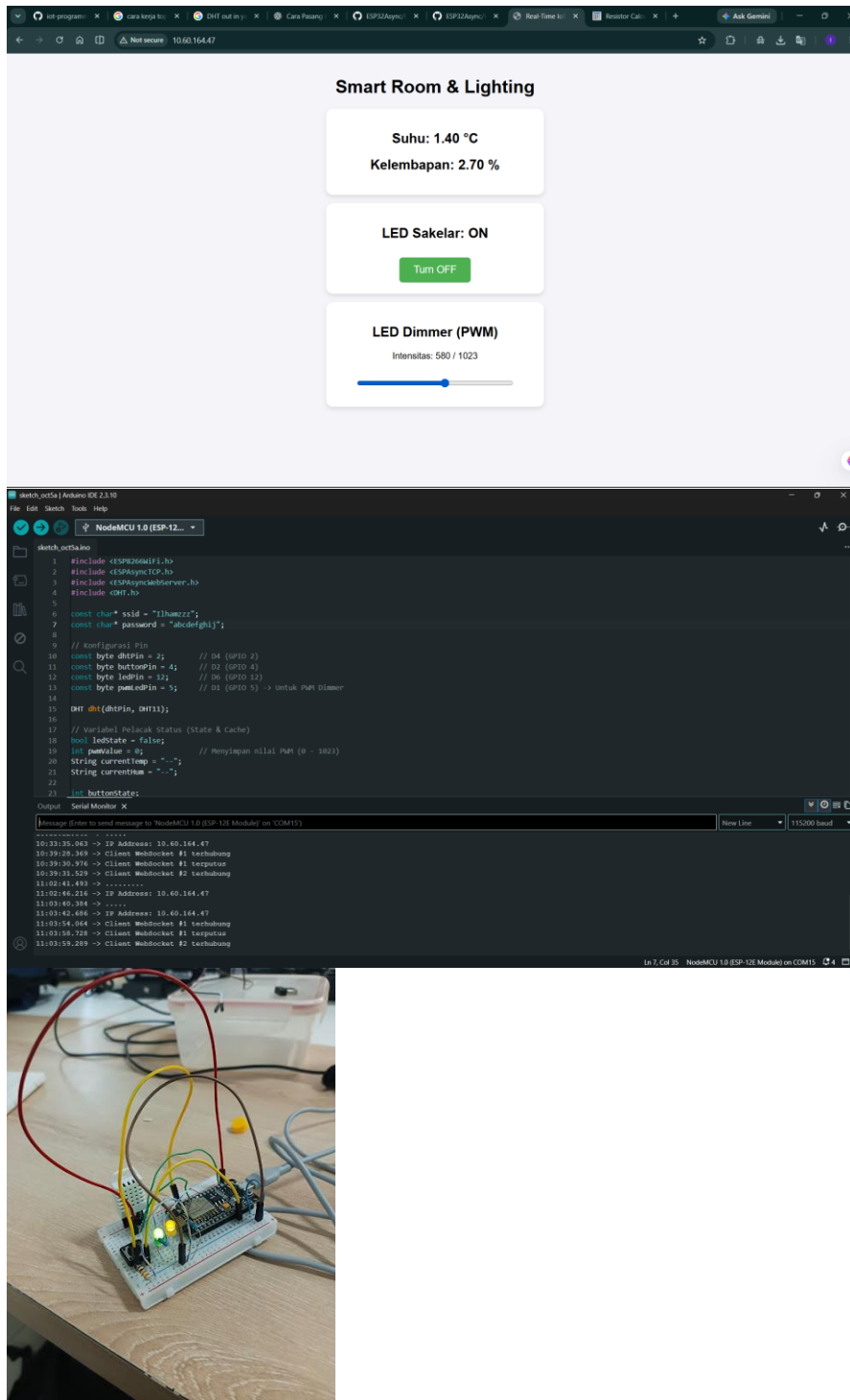
    // Ambil data kelembapan lalu tampilkan ke id humValue
    if(dataObj.hum !== undefined) {

```

```
document.getElementById('humValue').innerHTML = dataObj.hum;  
}
```

```
if(dataObj.led !== undefined) {  
    var btn = document.getElementById('toggleBtn');  
    var status = document.getElementById('ledStatus');  
    if(dataObj.led == "1"){  
        status.innerHTML = "ON";  
        btn.innerHTML = "Turn OFF";  
        btn.className = "btn-on";  
    } else {  
        status.innerHTML = "OFF";  
        btn.innerHTML = "Turn ON";  
        btn.className = "btn-off";  
    }  
}  
}  
}
```

4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*



Berikut langkah-langkah sederhananya:

- Rakit Komponen di Breadboard**
 - Pasang NodeMCU, sensor DHT (suhu/kelembapan), LED, dan push button.
 - Hubungkan pin VCC dan GND komponen ke 3V3 dan GND NodeMCU.
 - Hubungkan pin data sensor ke D4, LED ke D1, dan push button ke D5.
- Siapkan Arduino IDE**
 - Buka Arduino IDE di laptop.

- Pastikan library **ESPAsyncWebServer**, **ESPAsyncTCP**, dan **DHT sensor library** sudah terinstall.
- 3. **Upload Kode ke NodeMCU**
 - Salin kode program C++ yang sudah dibuat.
 - Ganti ssid dan password sesuai WiFi / Hotspot kamu.
 - Tancapkan kabel USB dari NodeMCU ke laptop, pilih port yang sesuai, lalu tekan tombol **Upload**.
- 4. **Cek IP Address**
 - Buka **Serial Monitor** di Arduino IDE (set ke 115200 baud).
 - Catat alamat IP yang muncul setelah NodeMCU berhasil konek ke WiFi (contoh: 192.168.1.15).
- 5. **Uji Coba di Web Browser**
 - Buka browser di HP atau laptop yang terhubung ke WiFi yang sama.
 - Ketik alamat IP NodeMCU tadi di address bar.